Practical 12

1.

AIM :

1. Word frequency: split a paragraph into words, count each in a HashMap, and print the

words sorted by frequency (highest first) using a Comparator.

CODE :

```
import java.util.*;


public class WordFrequency {     public
static void main(String[] args) {


        String paragraph = "java is easy java is powerful
java";
        String[] words =
paragraph.toLowerCase().split("\\s+");


        HashMap<String, Integer> frequency = new
HashMap<>();


        for (String word : words) {
```

```
            frequency.put(word,
frequency.getOrDefault(word, 0) + 1);
        }

        List<Map.Entry<String, Integer>> list =
new ArrayList<>(frequency.entrySet());

        list.sort((a, b) ->
b.getValue().compareTo(a.getValue()));

System.out.println("Word Frequency:");

        for (Map.Entry<String, Integer> entry : list) {
            System.out.println(entry.getKey() + " : " +
entry.getValue());
        }
    }
}
```

OUTPUT :

```
Word Frequency:
java : 3
is : 2
powerful : 1
easy : 1
```

2.

AIM :

Producer–consumer: one thread adds numbers to a small shared buffer and another

removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

CODE :

```
import java.util.*; public class
UniqueVisitors { public static void
main(String[] args) {     List<String>
logins = Arrays.asList(
        "Aesha", "Krisha", "Grishma",
        "Aesha", "Panthi", "Krisha", "Srushti"
   );
   // Count unique users using HashSet
```

```
    HashSet<String> uniqueUsers = new
HashSet<>(logins);


    // Maintain first-seen order using LinkedHashSet
    LinkedHashSet<String> orderedUsers = new
LinkedHashSet<>(logins);


    // Print unique user count
    System.out.println("Unique Visitors: " +
uniqueUsers.size());


    // Print first three distinct names
    System.out.println("First Three Distinct Names:");


    int count = 0;


    for (String name : orderedUsers) {
if (count == 3) {            break;
        }
        System.out.println(name);
count++;
    }
```

}

}

OUTPUT :

```
Unique Visitors: 5
First Three Distinct Names:
Aesha
Krisha
Grishma
```

3.

AIM :

(For Advanced Learners – home) Build a leaderboard from player scores held in a Map

and print the top-N sorted by score (ties broken by name); choose the data and sorting details.

CODE :

import java.util.*; public

class Leaderboard { public

static void main(String[]

args) {

```
    // Store player names and scores in a Map
Map<String, Integer> scores = new HashMap<>();

    scores.put("Aesha", 95);
scores.put("Krisha", 85);
scores.put("Grishma", 95);
scores.put("Panthi", 75);
scores.put("Srushti", 90);

    // Convert Map entries into a List
    List<Map.Entry<String, Integer>> players =
new ArrayList<>(scores.entrySet());

    // Sort by score (highest first), then by name
alphabetically     players.sort(
          Comparator.<Map.Entry<String, Integer>,
Integer>
                comparing(Map.Entry::getValue)
                .reversed()
                .thenComparing(Map.Entry::getKey)
    );
```

```
    // Print the top 3 players
int topN = 3;


    System.out.println("Top " + topN + " Players:");


    for (int i = 0; i < Math.min(topN, players.size());
i++) {
        System.out.println(
                players.get(i).getKey() + " : "
                        + players.get(i).getValue()
        );
    }
}
}
```

OUTPUT :

```
Top 3 Players:
Aesha : 95
Grishma : 95
Srushti : 90
```